

Audit Trail & Activity History

1. Feature Name

Audit Trail & Activity History

2. Purpose

- **Problem Solved:** Compliance and accountability require an immutable, searchable record of who did what, when, and on which entity.
 - **Why It Exists:** Every important mutation across the platform (auth, accounts, paths, classrooms, quizzes, community, assignments, AI feedback) writes an audit row. Users see **their own** history; the Super Admin alone can browse everyone's.
 - **Role in Application:** It is a cross-cutting, append-only observability layer. The `AuditLog` entity is deliberately immutable (private setters, no update/delete endpoints), and ownership/privilege checks are enforced inside the service, not just by route attributes.
-

3. What the User Can Do

Any authenticated user

- View **their own** append-only activity history (`GET /api/v1/history/me`), filtered by action type, date range, and text search, with pagination.
- Fetch the list of valid action types + their category labels for filter dropdowns.

Super Admin only (route requires `Admin` role + flag re-check)

- View **everyone's** history (`GET /api/v1/history/admin`) with additional filters by user, role, entity type, action, dates, search; sort newest-first or oldest-first.
 - See whether each referenced entity still exists (`EntityExists`), resolved per page.
 - Fetch the distinct entity-type list for filters.
 - Query `GET /api/v1/history/admin/access` to discover whether the current Admin is a Super Admin.
 - No write/update/delete endpoints exist for history (append-only).
-

4. Feature Workflow

[User performs an action] (e.g., completes a `module`, posts a comment)

- service calls `IAuditLogService.LogAsync(action, entityType, entityId, desc, old, new, extra)`
- `AuditLogService.ResolveActor` fills `UserId/Username/Role` from the JWT when not passed
- new `AuditLog` row inserted (`Timestamp =.UtcNow, immutable`)

[User views own history]

- `GET /api/v1/history/me?actionType=&fromDate=&toDate=&search=&page=&pageSize=`
- query hard-filtered to `a.UserId == userId` (enforced inside service)
- filters applied, paginated, newest-first

[Super Admin views all]

- `GET /api/v1/history/admin` → `EnsureSuperAdminAsync` (flag `check on every call`)
- optional `userId/role/entityType/actionType/date/search` filters
- `ResolveEntityExistenceAsync` batches existence checks per entity type

5. How It Works Internally

Actor Resolution (`ResolveActor`)

If `userId/username/role` aren't supplied, they are pulled from the current `HttpContext.User: NameIdentifier`, then `Email` (fallback to `NameIdentifier`), then the first `Role` claim.

Immutable Entity (`AuditLog`)

- `Id` (long), `Timestamp` (`UtcNow`), `UserId`, `Username`, `Role`, `ActionType` (enum), `EntityType`, `EntityId`, `Description`, `OldValue`, `NewValue`, `AdditionalData`.
- All setters are `private` — entries cannot be mutated after construction; there is no update/delete API.

Action Catalog (`AuditActionCatalog`)

- `AuditAction` enum holds 91 named actions (`LOGIN`, `ROLE_CHANGED`, `MODULE_COMPLETED`, `QUIZ_PASSED`, `POST_CREATED`, `REPORT_RESOLVED`, `SUBMISSION_GRADED`, `AI_FEEDBACK_GENERATED`, ...).
- `GetCategory` classifies actions into categories: **Authentication**, **Account**, **Academic**, **Quiz**, **Community**, **Admin**, **Assignment** (etc.).
- `GetActionName` / `GetOptions` supply labels/options for the UI filter dropdowns.

My History (`GetMyHistoryAsync`)

- Ownership enforced in the query: `.Where(a => a.UserId == userId)` — cannot be widened by callers.
- Pagination: `page ≥ 1`, `pageSize` default 20, clamped to `≤ 100`.
- Filters: `ActionType` `exact`, `FromDate` (start of UTC day), `ToDate` (end of UTC day, exclusive `<`), `Search` (substring on `Description`).
- Sort newest-first (Timestamp desc, Id desc).

Admin History (`GetAdminHistoryAsync`)

- `EnsureSuperAdminAsync` checks `Users.IsSuperAdmin` on **every** call (route `[Authorize(Roles = "Admin")]` is only a first gate).
- Adds filters for `UserId`, `Role`, `EntityType` (exact matches).
- `Sort=oldest` flips to ascending; anything else keeps newest-first.

Entity Existence (`ResolveEntityExistenceAsync`)

- For the current page only, batches referenced entity IDs per type (one query per type).
- Users are matched by string GUID; numeric types (`LearningPath`, `Classroom`, `Group`, `Post`, `Quiz`) by parsed int IDs.
- Unparseable/missing types → not resolved (`EntityExists = null`); resolved → `true/false`.
- Lets the UI show "deleted entity" for rows whose target was later deleted.

Producers (74 call sites)

Auth (`AUTH_REGISTERED`, `LOGIN`, `LOGIN_FAILED`, `LOGOUT`, `PASSWORD_CHANGED`), `UserService` (`PROFILE_UPDATED`, `USERNAME_CHANGED`, `EMAIL_CHANGED`), `LearningPath/Module` (created/updated/published/unpublished/archived/dependency/reorder), `Progress` (`MODULE_COMPLETED`, `CERTIFICATE_GENERATED`), `Attempt` (`QUIZ_STARTED/COMPLETED/PASSED/FAILED`), `Quiz/QuestionBank` (created/updated/published/archived/uploaded/versioned), `Classroom` (created/joined/left/member-removed/updated/deleted), `Community` (post/comment/vote/group/report/pin/unpin/ban), `Admin` (role/status/deletion/cert/classroom), `AiFeedback` (`AI_FEEDBACK_GENERATED`).

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/admin/AdminHistoryPage.tsx</code>	Super Admin audit viewer (filters, sort, pagination, entity-exists indicator)

Layer	File Path	Responsibility
Page	user history view (under profile/activity)	"My activity" list for the current user

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/HistoryController.cs	api/v1/history/* (me, action-types, admin, admin/access, admin/entity-types)
Service	backend/Services/Audit/AuditLogService.cs	Log writes, my-history, admin-history, actor resolution, existence resolution
Service	backend/Services/Audit/AuditActionCatalog.cs	Action → category/name presentation metadata
Interface	backend/Interfaces/Services/IAuditLogService.cs	Contract

8. Database Implementation

Entity/Table	Purpose
AuditLogs	Append-only audit rows (id long, Timestamp, actor UserId/Username/Role, ActionType enum, EntityType/Id, Description, Old/New values, AdditionalData)

- Migration: backend/Migrations/20260731103830_AddAuditLog.cs .

9. Security & Authorization

- All history endpoints `[Authorize]`; my-history only returns rows owned by the JWT user (enforced in the query, not the route).
 - Admin history additionally requires the `Admin` role **and** the `IsSuperAdmin` flag on every call (`EnsureSuperAdminAsync`) — a plain Admin is rejected with `UnauthorizedAccessException`.
 - Append-only: no route accepts a target user id, and there is no write/update/delete endpoint.
 - `AuditLog` immutability is structural (private setters) — even a compromised service can't edit a row.
-

10. Important Business Rules

1. **Append-only** — audit history cannot be edited or deleted through any API.
 2. **My-history ownership** is enforced server-side; callers can't pass another user's id.
 3. **Super Admin-only** for global history; the `IsSuperAdmin` flag is re-checked on every call.
 4. Date filters are day-granular (UTC): `FromDate` = start of day, `ToDate` = end of day (exclusive).
 5. Search is a case-insensitive substring of the description.
 6. Page size default 20, max 100; sort defaults to newest-first.
 7. `EntityExists` is resolved only for the current page, one batched query per entity type; `null` when not navigable.
 8. Actor identity falls back to the HTTP context claims when the caller omits it.
-

11. Example of Internal Execution

Step 1: An action is logged

- A user completes a module; `ProgressService` calls `_auditLog.LogAsync(MODULE_COMPLETED, "Progress", progress.Id, "Completed module: Variables 101")`.
- No actor passed → resolved from the JWT (`userId`, `email`, `role=Student`). Row inserted with `Timestamp =.UtcNow`.

Step 2: User views own history

- `GET /api/v1/history/me?page=1&pageSize=20` → query is `.Where(a => a.UserId == myId)`, newest-first, paginated; returns e.g. 12 entries with mapped `ActionName / Category`.

Step 3: Super Admin investigates a user

- `GET /api/v1/history/admin?userId=abc&sort=oldest` → `EnsureSuperAdminAsync` passes; filters by user, ascending order; existence resolver reports whether the referenced module/path still exists.

Step 4: Plain Admin is blocked

- An Admin (not Super) calls `GET /api/v1/history/admin` → flag check fails → `UnauthorizedAccessException` → 403.

12. Files Involved

Frontend

- `frontend/src/pages/admin/AdminHistoryPage.tsx`
- User history view (profile/activity page)

Backend

- `backend/Controllers/HistoryController.cs`
- `backend/Services/Audit/AuditLogService.cs`
- `backend/Services/Audit/AuditActionCatalog.cs`
- `backend/Interfaces/Services/IAuditLogService.cs`
- `backend/DTOs/Audit/AuditLogDto.cs`

Database

- `backend/Entities/AuditLog.cs`
- `backend/Entities/AuditAction.cs`
- `backend/Migrations/20260731103830_AddAuditLog.cs`

Producers (74 call sites across services)

- `backend/Services/Auth/AuthService.cs`, `User/UserService.cs`,
`LearningPath/LearningPathService.cs`, `Progress/ProgressService.cs`,
`Attempt/AttemptService.cs`, `Quiz/QuizService.cs`, `QuestionBank/QuestionBankService.cs`,
`Classroom/ClassroomService.cs`, `Community/CommunityService.cs`, `Admin/AdminService.cs`,
`Ai/AiFeedbackService.cs`, `backend/Controllers/AdminController.cs`

13. Functionality

- Append-only audit log for 91 action types across all domains
- Automatic actor capture from JWT (user id, email, role)
- My-history endpoint (ownership-enforced) with action/date/search filters + pagination
- Action-type options + category labels for filter dropdowns
- Super-Admin global history with user/role/entity-type/date/sort filters
- Per-page entity-existence resolution (batch queries per entity type)
- Entity-type list endpoint for filters
- Super-Admin access probe endpoint
- Structural immutability (private setters, no update/delete API)